

Phishing Website Detection with Machine Learning: A Critical Reproduction Study

Final Project — Data Science in Cyber (Dr. Uri Itai)

Submitted by: [Bshara Haj] — ID: [212590186]

Project repository: [<https://github.com/bsharahaj/phishing-detection-reproduction-study>]

Source under evaluation: N. Papernot, "Training a decision tree classifier to detect websites used for phishing" (tutorial + code: github.com/npapernot/phishing-detection)

Executive Summary

This project critically reproduces a widely-circulated tutorial that trains a decision tree to detect phishing websites and reports 90.5% accuracy. Using the UCI Phishing Websites dataset shipped inside the tutorial's own repository (11,055 sites, 30 pre-extracted categorical features), we first reproduced the author's experiment exactly as published, then rebuilt the entire pipeline to professional standards and measured how each of his methodological shortcuts affects the result.

The headline claim reproduces: on the author's own train/test split his decision tree reaches 90.4% accuracy, matching the published 90.5%. However, the number is weak evidence for three reasons. First, the code no longer runs (Python 2) and its documentation contradicts its behavior: comments describe a 456-row test set, while the shipped code actually tests on 9,055 rows and trains on only 2,000. Second, the experimental design leaks information: 47.1% of the dataset consists of duplicated rows, and 29.6% of the author's test rows are exact copies of training rows, so a substantial part of his test measures memorization rather than generalization. Third, accuracy is his only metric, on a dataset whose 44% phishing prevalence is far from any real-world deployment, and no seed, validation, EDA, or even a definition of the label is provided.

Our reproduction study finds the claim is untrustworthy rather than simply high or low: his design contains two opposing flaws — duplicate leakage inflates the number while his tiny ordered training slice deflates it — and the deflation dominates, so his 90.4% actually lands below the honestly achievable figure. Given a proper stratified split on the full data, his own model family reaches ~97% accuracy (his tiny ordered training slice handicapped it); but after removing duplicate leakage and 357 contradictory rows (identical feature vectors labeled both phishing and legitimate — making perfect accuracy mathematically impossible), the honest figure for a decision tree is 94.5%, and 96.3% for a Random Forest (MCC 0.927, ROC-AUC 0.994, 5-fold cross-validated F1 0.965 ± 0.004). Error analysis shows the model's blind spot is adversarially relevant: the phishing sites it misses are those with valid SSL certificates and well-formed page structure — exactly the direction real attackers have moved since free certificates became universal.

Verdict: the tutorial is honest about its number but not about what the number means. It is usable as a first classroom exercise, but its methodology should not be imitated for research or production. Our recommendation for similar problems is the pipeline built here: deduplication and contradiction removal, stratified seeded splits, multiple models, imbalance-aware metrics (MCC, precision/recall), and explicit error analysis.

1. Summary of the Source

Problem addressed. Phishing websites impersonate trustworthy sites to steal credentials and payment details. The tutorial addresses automatic detection: given features extracted from a website, classify it as phishing or legitimate. The problem matters because phishing remains one of

the most common initial-access vectors in real breaches, victimizes non-technical users directly, and operates at a scale (thousands of new sites daily) where manual blocklisting cannot keep up.

Proposed solution. Train a scikit-learn *DecisionTreeClassifier* with default hyperparameters on pre-extracted website features; a second script trains a logistic regression. The tutorial's stated result is 90.5% test accuracy for the tree.

Dataset. The repository ships *dataset.csv*: the UCI “Phishing Websites” dataset (Mohammad, Thabtah & McCluskey, 2015) — 11,055 rows, 30 features plus a label, with every value encoded as -1 (suspicious), 0 (borderline) or 1 (legitimate-looking). Features cover URL lexical properties (IP address in URL, length, '@' symbol, prefix-suffix dashes), page structure (anchor URLs, iframes, server-form handlers), and external reputation (web traffic rank, DNS record, domain age, Google indexing). Notably, the tutorial never states which label value means phishing; we had to consult the UCI documentation (-1 = phishing).

Methodology employed by the author. Load the CSV with numpy; split by taking the first 2,000 rows as training and the remainder as test (no shuffling, no stratification, no seed); train with default parameters; report accuracy as the single metric. No EDA, no preprocessing, no validation, no error analysis.

2. Critical Evaluation

Main claims. (1) A simple decision tree detects phishing websites with 90.5% accuracy; (2) implicitly, that the presented procedure is a sound way to build and measure such a detector.

Is claim 1 supported? Numerically, yes: rerunning his exact pipeline (after a Python-3 port) yields 90.4% — within run-to-run variance of the published 90.5% (the author fixed no random seed, so his own result varies slightly from run to run). We consider the headline number honestly reported.

Is the evaluation methodology appropriate? No, on five counts. (a) **Ordered split:** the first-2,000-rows split is neither shuffled nor stratified; the code comment claims the test set contains 456 websites while the shipped data produces 9,055 — the code was evidently written for a different (smaller) version of the dataset and never updated. (b) **Duplicate leakage:** 47.1% of rows are duplicates, and 2,680 of the 9,055 test rows (29.6%) are exact copies of training rows. Nearly a third of the reported “test” therefore measures memorization. (c) **Single metric:** accuracy alone hides the false-positive/false-negative structure that defines the cost of a security system, and is misleading when the dataset's class balance (44% phishing) bears no relation to deployment prevalence. (d) **No validation or stability control:** a single unseeded run, no cross-validation. (e) **Undefined label and undated data:** the label semantics are never stated, and the absence of timestamps precludes any assessment of robustness to the evolution of phishing tactics.

Weaknesses beyond methodology. The feature-extraction code that turned raw URLs into the CSV is not part of the repository, so the pipeline cannot be applied to any new website — the detector cannot actually be deployed from what is published. The strongest features have also aged: *SSLfinal_State* was highly discriminative when certificates were costly, but free certificate authorities have since made valid HTTPS standard on phishing sites.

Are the conclusions justified? The narrow numeric claim, yes. The implicit conclusion — that this is how one should build and evaluate a phishing detector — is not. Our experiments (Section 5) show the same number can be produced by an under-trained model measured on a leaky test set, which is not evidence of a working method. Where our findings contradict the author is therefore not the number itself but its meaning.

3. Feature Engineering Analysis

Was feature engineering performed by the author? None. Values flow from CSV to model untouched. This is partially defensible only because the dataset's creators performed the substantive engineering beforehand — converting raw URLs, HTML and reputation lookups into ternary codes. That step is a hidden, non-reproducible transformation: its thresholds (e.g. what URL length maps to -1) are documented only in the original UCI paper and its code is absent.

Transformations we applied, and why. (1) *Label mapping* to phishing = 1 (positive class), so precision/recall carry their conventional detection meaning. (2) *No scaling*, with justification: all features already share the [-1, 1] range, and tree models are scale-invariant; applying standardization would be transformation for its own sake. (3) *Encoding experiment*: the codes are ordinal by design ($-1 < 0 < 1$ in “suspiciousness”), but for a linear model this ordering is an assumption. We trained logistic regression on both the raw ordinal matrix and a one-hot expansion: one-hot won clearly (accuracy 93.8% vs 91.3%, MCC 0.876 vs 0.825), showing the ordinal assumption is wrong for at least some features and costs a linear model real performance. Tree models are unaffected. (4) *Deduplication and contradiction removal*: we removed 357 rows whose feature vectors appear with both labels (irreducible label noise) and collapsed exact duplicates, leaving 5,721 unique vectors; this is the transformation with the largest effect on measured performance, because it eliminates train-to-test leakage. (5) *Feature creation*: `n_red_flags`, the count of suspicious (-1) indicators per site — the “many weak signals” intuition of a security analyst. Phishing sites average 10.4 flags versus 7.7 for legitimate ones (Spearman $\rho = 0.39$ with the label); its incremental value on top of tree models proved marginal, as expected, since trees already combine individual flags.

Redundancy in the system. We spot it two ways: identical-column checks and a Spearman correlation scan of all feature pairs. `Favicon` and `popUpWidnow` correlate at $\rho = 0.94$ — nearly duplicate signals — and several pairs exceed 0.75. We tackled it by dropping, from each highly-correlated pair, the feature less associated with the label, plus five near-zero-signal features (e.g. `RightClick`, `Iframe`), yielding a 23-feature set. Retraining on the reduced set changed Random Forest accuracy by less than half a point (96.5% vs 96.3%) — confirming the dropped features were indeed redundant: their information was already carried by the rest.

Was the (absent) feature engineering meaningful, and what could improve performance? Mathematically, the dataset's pre-encoding compresses continuous signals into three levels, losing information a model could use (e.g. the actual domain age in days rather than a binary flag). From the cybersecurity side, the most valuable additional features would be ones harder for an attacker to fake: certificate issuer and age (not mere presence), passive-DNS history, hosting ASN reputation, URL character n-grams or entropy, page-content similarity to known brands, and — critically — collection timestamps to allow temporal validation.

4. Reproducibility Analysis

Can the code be executed? Not as published: both scripts use Python 2 print statements and die with a `SyntaxError` on any modern interpreter. After a minimal mechanical port (print functions only), both run.

Are all files and dependencies available? Yes — the dataset ships in the repository and the only dependencies are `numpy` and `scikit-learn`. This is the tutorial's genuine strength; many sources fail here.

Hidden preprocessing. Two layers: (1) the raw-URL-to-features extraction performed by the dataset's creators, absent from the repository, undocumented in the tutorial, and irreproducible today (reputation features like `web_traffic` referenced services that have since changed or disappeared); (2) the dataset mismatch — the code's comments and the published accuracy correspond to a 2,456-row version of the data, while the repository ships 11,055 rows, silently changing what the code measures.

Overall verdict. Mixed. Data and dependencies: excellent. Code: dead but trivially revivable. Documentation-to-code consistency: poor. End-to-end reproducibility of the deployed system (raw website in, verdict out): impossible from what is published. The headline number reproduces (90.4% vs 90.5%) — but partly by luck, on a test set four times larger than the author believed he was using.

5. Experimental Results

Design. Three model families (Decision Tree — the author's; Random Forest, 200 trees; Logistic Regression) were each trained under five controlled configurations, so that every methodological decision is isolated as a measurable experiment. All our runs use seed 42; the clean configurations use a stratified, shuffled 80/20 split.

Config	Data	Split	Isolates the effect of
A	full 11,055 rows	author's: first 2,000 train, rest test	— (baseline reproduction)
B	full 11,055 rows	stratified 80/20	the author's split (A vs B)
C	5,721 unique vectors	stratified 80/20	duplicate leakage (B vs C)
D	as C, 23 selected features	stratified 80/20	redundancy removal (C vs D)
E	as C + <code>n_red_flags</code>	stratified 80/20	engineered feature (C vs E)

Results (test-set metrics; phishing = positive class):

Config	Model	Acc.	Prec.	Recall	F1	F2	MCC	ROC-AUC
A author-split	Decision Tree	0.904	0.940	0.837	0.886	0.856	0.807	0.909
A author-split	Random Forest	0.931	0.962	0.878	0.918	0.894	0.861	0.979
A author-split	Logistic Regr.	0.845	0.916	0.716	0.804	0.749	0.692	0.930
B shuffled-full (still leaky)	Decision Tree	0.970	0.976	0.955	0.965	0.959	0.939	0.976
B shuffled-full (still leaky)	Random Forest	0.977	0.980	0.967	0.974	0.970	0.953	0.996
B shuffled-full (still leaky)	Logistic Regr.	0.928	0.934	0.900	0.917	0.907	0.853	0.978
C dedup (honest)	Decision Tree	0.945	0.946	0.948	0.947	0.947	0.890	0.945
C dedup (honest)	Random Forest	0.963	0.974	0.954	0.964	0.958	0.927	0.994
C dedup (honest)	Logistic Regr.	0.913	0.918	0.912	0.915	0.913	0.825	0.977
D dedup-reduced	Random Forest	0.965	0.978	0.954	0.966	0.959	0.930	0.994

E dedup+red_flags	Random Forest	0.964	0.974	0.956	0.965	0.959	0.929	0.994
----------------------	---------------	-------	-------	-------	-------	-------	-------	-------

Reading the table. A vs B: a shuffled split and full training data lift the author's own model from 90.4% to 97.0% — but B is still contaminated by duplicates, so this 97.0% is not the honest figure; it simply shows his ordered 2,000-row slice under-trained the model. The trustworthy result is config C below. B vs C: removing leakage costs about 1.5–2.5 points across models; that gap is the measured size of the duplicate-inflation. Config C is the honest setting: Decision Tree 94.5%, Random Forest 96.3% (MCC 0.927). D and E confirm that redundancy removal is harmless and the engineered feature adds little to tree models. Logistic regression on one-hot encoding (not in the table) reached 93.8% accuracy / MCC 0.876 on config C, versus 91.3% / 0.825 with ordinal coding.

We additionally report F2 ($F\beta$ with $\beta = 2$), which weights recall twice as heavily as precision — the appropriate emphasis for phishing detection, where a missed phishing site (false negative) is costlier than a false alarm. On config C the Random Forest reaches $F2 = 0.958$.

Stability: 5-fold cross-validated F1 of the Random Forest on the clean data is 0.965 ± 0.004 .

Error analysis. The headline model errs on 42 of 1,145 clean test rows: 15 false positives, 27 false negatives (FP rate 2.7%, FN rate 4.6%). The pattern is systematic, not random: missed phishing sites tend to carry valid SSL certificates and well-formed anchor structure — attackers who invest in looking legitimate — while false positives concentrate on legitimate sites with poor hygiene (no traffic rank, odd subdomains), typically small or new businesses. Security implications: the FN pattern is the adversarial direction phishing has actually moved (free certificates), so real-world recall would degrade over time; the FP pattern imposes economic harm on precisely the sites least able to contest blocking. The FP/FN balance is a policy choice, not a model property: moving the decision threshold from 0.5 to 0.3 raises recall from 95.4% to 97.8% at the cost of precision (97.2% → 92.6%), while 0.7 yields 98.9% precision at 91.2% recall. A browser-warning deployment (cheap FP) would sit low; an automated-takedown system (costly FP) high. The author's single accuracy number cannot support this conversation at all.

6. Conclusions

Key findings. (1) The published 90.5% reproduces (90.4%) but on a broken experimental design: dead code, documentation describing a different dataset, an ordered unstratified split, and 29.6% test-set leakage. (2) The dataset itself is flawed in instructive ways: 47.1% duplicate rows and 357 contradictory rows that make perfect accuracy impossible. (3) On a clean, leakage-free setup the task remains learnable: Random Forest reaches 96.3% accuracy, MCC 0.927. (4) A handful of features dominate (SSLfinal_State $\rho = 0.74$, URL_of_Anchor $\rho = 0.70$) — the dataset is “easy,” so high accuracy is a low bar and not by itself evidence of methodological quality. (5) The model's failure mode aligns with the real evolution of phishing, which limits the shelf life of any detector built on these features.

Lessons learned. Reproducing a number is not validating a method; the two can diverge completely. Sanity-check assertions embedded in the pipeline caught a subtle deduplication bug (contradictory labels) that manual inspection missed. Metric choice is a security decision: accuracy conceals exactly the information (FP/FN structure) that determines whether a detector is deployable.

Strengths of the original work: self-contained data, minimal dependencies, clear pedagogical intent, line-by-line commented code, and an honestly-reported number. **Weaknesses:** everything methodological — split design, leakage, single metric, no validation, no EDA, no error analysis, unmaintained code, and silent divergence between documentation and data.

Future improvements. Re-extract features from raw, timestamped URL feeds (PhishTank + benign crawls) so temporal validation becomes possible; replace aged reputation features with harder-to-fake signals (certificate issuer/age, passive DNS, hosting ASN); evaluate under realistic prevalence with precision-recall analysis; add adversarial evaluation (perturbing attacker-controllable features); and calibrate the decision threshold to an explicit FP/FN cost model of the target deployment.

7. Summing It Up

We evaluated a popular decision-tree phishing-detection tutorial against its own claims. Problem: automatic phishing website classification. Source: Papernot's tutorial and repository. Data: UCI Phishing Websites (11,055 rows, 30 ternary features) as shipped in that repository. Method: exact reproduction of the author's pipeline, followed by a rebuilt pipeline (contradiction removal, deduplication, stratified seeded splits, three model families, seven metrics, error analysis) in which each of the author's shortcuts was isolated and measured. Main findings: the 90.5% claim reproduces numerically (90.4%) yet rests on an ordered split with 29.6% test leakage, a dataset that is 47.1% duplicates, and a single prevalence-sensitive metric; the honest performance on clean data is 94.5% (his model) to 96.3% (Random Forest, MCC 0.927, CV-stable). The most important insights: reproduction of a number is not validation of a method; duplicate and contradictory rows silently corrupt evaluation; and this feature set's blind spot (SSL-equipped, well-formed phishing sites) is exactly where attackers have moved. Recommendation: suitable as a first classroom exercise only; for any similar problem, use the leakage-aware pipeline demonstrated here rather than the tutorial's procedure. Final conclusion: the author's claim is numerically supported but methodologically hollow — a precise measurement of the wrong experiment.